SIMATS ENGINEERING

CO1 - ASSESSMENT TOOL 2

Error Analysis Task

COURSE NAME: Deep Learning COURSE CODE: MLA0403
FACULTY : Dr.Arul Raj A.M

COURSE OUTCOME COVERED

CO 1: Learning Algorithms, Maximum Likelihood Estimation (MLE), Building Machine Learning Algorithms, Neural Networks, Artificial Neurons, Perceptron, Multilayer Perceptron (MLP), Forward Propagation, Backpropagation Algorithm, Gradient Descent, Stochastic Gradient Descent (SGD), Mini-Batch Gradient Descent, Curse of Dimensionality. (BTL-4)

CO1 Weightage: 15%

Assessment Tool Weightage: 30%

Duration: 30 minutes

Marks: 25

Code of Conduct:

I, M Sohail Khan (192525046) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
M Sohail Khan

QUESTIONS: **Total Marks:25**

1. Error Analysis in Maximum Likelihood Estimation (MLE)

A student builds a binary classifier using Maximum Likelihood Estimation. The likelihood function is incorrectly written as the sum of probabilities instead of the product. The model produces poor parameter estimates even with large data.

Tasks:

a) Identify the conceptual error in the likelihood formulation.
b) Explain how this mistake affects parameter estimation.
c) Suggest the correct mathematical formulation and reasoning.
d) How would using log-likelihood fix computational issues?

ANSWERS:

1. Error Analysis in Maximum Likelihood Estimation (MLE)

a) Conceptual Error:
The student incorrectly wrote the likelihood as the sum of probabilities instead of the product

of probabilities. Since the observations are assumed to be independent, the likelihood must be the product of the probability of each observation.
b) Effect on Parameter Estimation:
Using the sum changes the objective function, so the model no longer finds the parameters that maximize the true likelihood. As a result, the estimated parameters become inaccurate, even with a large amount of data.
c) Correct Mathematical Formulation:
For independent observations:
$L(\theta) = P(x_1, x_2, ..., x\square \mid \theta)$
$= P(x_1 \mid \theta) \times P(x_2 \mid \theta) \times ... \times P(x\square \mid \theta)$
$= \Pi P(x_i \mid \theta)$
The best parameter estimate is the value of θ that maximizes this likelihood.
d) Why Use Log-Likelihood?
The log-likelihood converts the product into a sum:
$\log L(\theta) = \Sigma \log P(x_i \mid \theta)$
This avoids numerical underflow when multiplying many small probabilities, simplifies differentiation, and makes optimization much easier.

2. Error Diagnosis in Perceptron Learning

While implementing a Perceptron, a student observes that the model fails to converge even after many iterations on a linearly separable dataset.
Tasks:
a) List possible implementation errors in the learning algorithm.
b) Analyze how incorrect learning rate or weight updates affect convergence.
c) Identify dataset-related issues that may cause this problem.
d) Propose modifications to ensure convergence.

ANSWER:

2. Error Diagnosis in Perceptron Learning
a) Possible Implementation Errors:

- Incorrect weight update rule.
- Bias term not included or updated.
- Wrong prediction function or threshold.
- Incorrect feature scaling.
- Programming mistakes in the training loop.

b) Effect of Incorrect Learning Rate or Weight Updates:
A very large learning rate can cause the weights to overshoot the correct solution, while a very small learning rate makes learning extremely slow. Incorrect weight update equations prevent the perceptron from reaching the correct decision boundary.
c) Dataset-Related Issues:
Although the dataset is said to be linearly separable, problems such as mislabeled samples, noisy data, duplicate records, or incorrect preprocessing may prevent convergence.
d) Modifications to Ensure Convergence:

- Verify the perceptron update rule.
- Include and update the bias term.
- Normalize the input features.
- Choose a suitable learning rate.
- Check for labeling or preprocessing errors.

With a correctly implemented algorithm and truly linearly separable data, the perceptron is guaranteed to converge.

3. Backpropagation Gradient Error Analysis

A Multilayer Perceptron trained using Backpropagation shows increasing loss during training instead of decreasing.

Tasks:
a) Identify possible mathematical or coding errors in gradient computation.
b) Explain the role of activation functions in gradient flow.
c) Analyze how vanishing or exploding gradients affect training.
d) Suggest techniques to fix the issue (e.g., normalization, initialization).

ANSWER:

3. Backpropagation Gradient Error Analysis

a) Possible Errors in Gradient Computation:

- Incorrect derivative calculations.
- Wrong sign in gradient updates.
- Incorrect implementation of the chain rule.
- Learning rate set too high.
- Bugs in backpropagation code.

b) Role of Activation Functions:

Activation functions introduce non-linearity, allowing the network to learn complex patterns. Their derivatives determine how gradients flow through the network during backpropagation.

c) Vanishing and Exploding Gradients:

Vanishing gradients become very small, causing early layers to learn very slowly. Exploding gradients become extremely large, leading to unstable weight updates and increasing training loss.

d) Techniques to Fix the Issue:

- Use ReLU or related activation functions.
- Apply Xavier or He weight initialization.
- Normalize inputs using Batch Normalization.
- Use gradient clipping.
- Reduce the learning rate if necessary.

These techniques improve gradient flow and stabilize training.

4. Gradient Descent vs SGD Error Investigation

A model trained using Gradient Descent converges very slowly, while the same model with Stochastic Gradient Descent shows unstable loss fluctuations.

Tasks:
a) Explain why batch gradient descent is slow in large datasets.
b) Analyze the cause of fluctuations in SGD.
c) Compare the error behavior of Mini-Batch Gradient Descent.
d) Recommend the best approach and justify with error trade-offs.

ANSWERS:

4. Gradient Descent vs SGD Error Investigation

a) Why Batch Gradient Descent is Slow:

Batch Gradient Descent computes gradients using the entire training dataset before each update. For large datasets, this requires more computation and results in slower training.

b) Cause of Fluctuations in SGD:

SGD updates the model using one training sample at a time. Since each sample is different, the gradient estimates are noisy, causing the loss to fluctuate during training.

c) Mini-Batch Gradient Descent:

Mini-Batch Gradient Descent uses a small group of samples for each update. It is faster than Batch Gradient Descent and produces smoother updates than SGD, making training more stable.

d) Best Approach:

Mini-Batch Gradient Descent is generally the preferred choice because it balances speed and stability. It trains faster than Batch Gradient Descent while reducing the large fluctuations seen in SGD, resulting in better overall convergence.

5. Curse of Dimensionality Error Analysis

A machine learning model built using high-dimensional data performs well on training data but poorly on test data due to overfitting, related to the Curse of Dimensionality.

Tasks:

a) Explain how high dimensionality increases model error.
b) Identify signs of overfitting in this scenario.
c) Analyze why distance-based learning fails in high dimensions.
d) Suggest dimensionality reduction or regularization techniques to improve performance.

ANSWERS:

5. Curse of Dimensionality Error Analysis

a) How High Dimensionality Increases Error:

As the number of features increases, the model becomes more complex and requires much more data to learn effectively. With limited data, it tends to fit noise instead of meaningful patterns, increasing prediction error.

b) Signs of Overfitting:

- Very high training accuracy.
- Low test accuracy.
- Large gap between training and testing performance.
- Poor generalization to new data.

c) Why Distance-Based Learning Fails:

In high-dimensional spaces, the distances between data points become very similar. As a result, algorithms such as K-Nearest Neighbors (KNN) cannot effectively distinguish between close and distant points, reducing prediction accuracy.

d) Techniques to Improve Performance:

- Use Principal Component Analysis (PCA) to reduce the number of features.
- Apply feature selection to remove irrelevant features.
- Use L1/L2 regularization to reduce model complexity.
- Collect more training data if possible.

These methods reduce overfitting, simplify the model, and improve generalization on unseen data.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to	Applies relevant concepts	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts

		solve complex problems	correctly with minor errors		
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand